ATTENDEASE — System Presentation Manuscript

Laboratory exam: System presentation · Web-based attendance management · Align with approved project documentation and live demo.

1. Introduction

System title

ATTENDEASE — Smart Attendance Management System

Members and roles

Complete the table below with your group's names and responsibilities.

Name	Role in group	System role (for demo)
[Name]	[e.g., Team leader, documentation]	[e.g., Superadmin / Instructor / Student]
…	…	…

Overview of the system

ATTENDEASE is a web application for managing classroom attendance. It provides **instructors** with tools to run sessions, manage classes, students, and schedules, and view records and reports (with administrator-level access for selected modules). **Students** can view their schedule, see attendance history, and check in using **QR code scanning**, with a manual fallback where implemented. The public **landing page** presents the product and supports login and sign-up flows. The system uses **role-based access** so each user type sees only the appropriate dashboard.

2. Problem Statement

What problem does the system solve?

Manual attendance (paper lists, spreadsheets, or informal messaging) is slow, error-prone, and difficult to audit. It is hard to track **present / late / absent** status in real time, produce consistent **reports**, and give students a clear view of their own attendance history.

Target users

- **Instructors / faculty** — take attendance, manage class and student data, review records.
- **Students** — check in for class (e.g., via QR), view schedule and personal attendance.
- **Administrators** (as implemented in routes) — broader access to reports and instructor accounts where `admin` / `superadmin` roles are used.

3. Objectives

General objective

To design and implement a web-based attendance management system that digitizes check-in, centralizes records, and supports reporting for educational settings.

Specific objectives

Adjust to match what you demonstrate and what your instructor approved.

1. Provide secure, **role-based** access for instructors, students, and admins.
2. Enable **QR-based** (and optional manual) attendance submission for students.
3. Allow instructors to maintain **classes, students, schedules**, and attendance sessions/records.
4. Support **reports and analytics** (and export, if shown in the demo).
5. Deliver a **responsive** UI for desktop and mobile browsers.
6. Persist data via a **REST-style API** (JSON Server for local development; mention Firebase only if the group actually uses it in the final system).

4. System Features

Core functionalities

- Landing page with feature highlights (QR, dashboards, reports, responsiveness).
- Authentication session with **route guards** by role: instructor, student, admin, superadmin.
- **Instructor area:** overview, attendance, records, students, classes, schedules, settings, account; instructor management, pending approvals, instructor accounts; reports and instructor-accounts restricted to admin/superadmin.
- **Student area:** overview, schedule, my attendance (QR/manual), QR code display, settings.
- QR utilities: generation/display and camera-based scanning (e.g., jsQR), per implementation.

Key modules

Module	Purpose
Landing & auth	Entry, login/sign-up, session
Instructor dashboard	Layout and navigation to instructor features
Attendance & records	Sessions, history, status
Classes & students	Roster and section management
Schedules	Timetable / subject metadata
Reports	Aggregated views (admin)
Student dashboard	Schedule, attendance submission, personal QR

5. System Design

Architecture

- **Client:** Single Page Application — Angular 21, standalone components, TypeScript.
- **API:** HTTP to backend — JSON Server serving `db.json` in development (REST JSON).
- **Cross-cutting:** RxJS, SweetAlert2, Angular Router and route guards.

UML — Use case (summary)

- **Student:** Log in, view schedule, submit attendance (QR/manual), view own records, QR profile as designed.
- **Instructor:** Log in, manage attendance, view/edit records, manage students/classes/schedules, account settings.
- **Admin / Superadmin:** Instructor capabilities where allowed, plus reports, instructor accounts, and approvals per routes.

ERD — Entities (`db.json`)

Represent as a diagram in your PPT; relationships described below.

- **students** — id, fullName, email
- **schedules** — id, day, time, subject, meta, mode
- **attendanceRecords** — id, subject, section, date, timeIn, status
- **instructorClasses** — id, name, program, yearLevel, studentCount, status
- **instructorStudents** — id, name, studentId, email, section
- **instructorSchedules** — id, day, time, title, meta, mode
- **instructorSessions** — id, subject, section, date, status
- **instructorAccounts** — id, fullName, email

Students belong to sections; **attendanceRecords** tie subject, section, and date to status; **instructorSessions** represent class meetings; schedules describe when subjects meet.

Optional diagrams

- Component or navigation diagram from `app.routes.ts` (landing → instructor/student layouts → feature pages).
- Sequence: student scans QR → decode → API request → record stored → confirmation.

6. Live Demonstration (suggested flow)

1. Landing page and features; log in as instructor, then as student (or two browsers).
2. Instructor: overview → classes / students → schedules → attendance (session).
3. Student: schedule → attendance → QR scan (or manual code) → confirm record.
4. Admin (if applicable): reports, instructor-accounts.
5. Navigation, logout; optional: wrong-role URL shows guard redirect.

Prepare seeded `db.json` data and test accounts before the exam.

7. Tools & Technologies Used

Category	Technology
Language	TypeScript
Frontend framework	Angular 21 (standalone components, Router)
UI	HTML, SCSS, SweetAlert2
QR	jsQR, qrcode
Data (development)	JSON Server, db.json
Other	RxJS, Node.js, npm, Angular CLI 21.2.x

Add Firebase or other services only if your group actually uses them in the final demo.

8. Challenges Encountered

Replace with your group's real experience. Examples:

- QR scanning across devices and browsers (permissions, localhost); mitigation: manual code and clear errors.
- Aligning client session storage with route guards; testing every role's allowed paths.
- Prototype backend (JSON Server) vs. production needs (server-side auth); document limitations.
- Keeping UI state consistent after attendance submission.

- Scope and time: prioritizing one complete attendance flow and core CRUD.

9. Conclusion & Recommendations

Conclusion

ATTENDEASE addresses the need for a centralized, role-based attendance system with QR check-in, instructor management features, and student self-service. The project demonstrates SPA development with a REST-style API suitable for pilot use in academic settings.

Recommendations

- Move to a production backend with server-side authentication and validation.
- Add automated tests and audit logs for attendance changes.
- Consider a mobile app or PWA with offline queue for poor connectivity.
- Align exports and reports with registrar or institutional formats.

> **PPT checklist (suggested):** Title · Introduction · Problem & users · Objectives · Features · Modules/sitemap · Architecture · Use case · ERD · Demo flow · Screenshots · Tech stack · Challenges · Conclusion · Q&A.